Yuxiang Jiang (u214577)
Isaac Pérez Anguiano (u214402)
Marc Rodriguez Rodriguez (u214013)

Part 3: Ranking & Filtering

3.1. Ranking

1. Explain how the ranking differs when using TF-IDF and BM25, and think about the pros and cons of using each of them. Regarding your own score, justify the choice of the score (pros and cons). HINT: Look into numerical fields that each record has to build your score.

TF-IDF with cosine similarity measures how important each word is by turning documents and queries into weighted word lists. It gives higher weight to rare terms and then compares the two vectors by looking at the angle between them. BM25, on the other hand, uses a probabilistic scoring formula that limits how much repeating a word can increase the score. It also prevents longer documents from getting an unfair advantage by using a length-normalization parameter called b, which controls how strongly document length should affect the score.

TF-IDF's main advantages are that it's simple, easy to use, and very fast, especially when you're working with small or fairly similar sets of documents. It's good at picking out unique or uncommon terms, and it works well with cosine similarity for short queries. But it still has some problems, even though cosine similarity uses a square-root length normalization, repeated words can still boost a document's score more than they should, and TF-IDF doesn't fully fix the bias that can appear with very short or very long documents. This can still cause some results to lean toward one side depending on the collection you're working with.

BM25 fixes many problems of TF-IDF by limiting the effect of repeated terms and adjusting more directly for document length. Because of this, it tends to perform better across a wide range of datasets and is a common choice in modern search engines. Its main drawbacks are that it needs a bit of tuning to get the best results, and like TF-IDF, it doesn't understand deeper meaning or context.

Your Score: Here, the task is to create a new score. (Be creative, think about what factors could make a document more relevant to a query and include them in your formula.)

For each term in a document:

$$\text{term\_score}_{t,d} = \frac{tf_{t,d}}{\text{doc\_len}_d} \cdot idf_t$$

Combine term score for a document

$$\text{score}_d = \frac{n}{\sum_{t \in Q} \frac{1}{\text{term\_score}_{t,d}}}$$

We take inspiration from TF-IDF and BM25 to design our ranking method. For each search query, the system first computes a TF-IDF–style score for every query term in each document. To prevent longer documents from having an unfair advantage, the term frequency is normalized by the document's length and then multiplied by the term's inverse document frequency. Instead of simply summing these scores, we combine them using the harmonic mean, which rewards documents where all query terms are well represented. Our expectation is that this approach ensures that rare but informative terms still carry weight through IDF, while the harmonic mean balances their contribution so that results reflect both the rarity of terms and all query terms are evenly represented throughout the document. Its main drawback is it doesn't understand deeper meaning or context and .

From the last part:
The description of some of the shoes products announce offers for other products which contain the words in the query. For example, when checking the JSON description of the product with PID SHOFUH52V5AVNHMN, it states "This season Rockfield has introduced sports shoes for both men and boys." This description is unrelated to the actual product and only matches the query by coincidence. As a result, Query 4 shows low precision and low numbers of relevant products retrieved (2 only). To address this issue, the retrieval system could implement text preprocessing and filtering steps to remove generic or promotional sentences that are not specific to the product itself.

This problem still remains unsolved for TF-IDF + cosine similarity but for BM25 and our ranking method is solved.

From the last part:
In addition, we have found that when using the query "black sports shoes", one of the retrieved results was a pair of sneakers that were entirely yellow with only small black details (as it can be seen in the image below). The product was tagged with the colors (yellow, black), which explains why it appeared in the search results. However, from a user's perspective, this result is not truly relevant, since the dominant color of the shoes is yellow rather than black. This highlights a limitation in the retrieval process, where the presence of a keyword in the metadata (in this case, "black") can lead to false positives when the attribute is not representative of the product's overall appearance. This issue is harder to address.

This problem still remains unsolved for all three rankings methods.

2. Implement word2vec + cosine ranking score. Return a top-20 list of documents for each of the 5 queries defined in the Part 2 of your project, using search and word2vec + cosine similarity ranking.
The queries of the part 2 were:
q1 = "casual half sleeve polo shirt for men", q2 = "light blue jeans slim fit", q3 = "trousers chino casual men", q4 = "black sports shoes", q5 = "fancy t-shirt".

3. Can you imagine a better representation than word2vec? Justify your answer. (HINT: what about Doc2vec? Sentence2vec? What are the pros and cons?)
Word2Vec is a widely-used method for generating word embeddings, but it has limitations: it produces only word-level vectors, it is context-independent, and document representations must be created manually by averaging word embeddings. These constraints often reduce its effectiveness in tasks involving larger sentences such as product descriptions.
Several alternatives provide richer, more expressive representations.

Doc2Vec extends Word2Vec by learning a single vector representation for an entire document rather than individual words.
Advantages:

- Captures some notion of the global meaning of the text.
- Can be trained on domain-specific contexts.

Limitations:

- It requires substantial training data.
- Adding new documents requires retraining.

Sentence embedding models are designed to represent entire sentences or short texts in a single semantic vector.

Advantages:

- Capture the sentence-level meaning much more effectively than averaging word embeddings.
- It allows strong performance without large training datasets.

Limitations:

- Higher computational cost compared to Word2Vec or Doc2Vec.
- Domain adaptation may require labeled training pairs for fine-tuning.

While Word2Vec is useful for capturing word-level similarities, it is not well-suited for representing complete sentences or documents (or products in our case). For these tasks, more effective alternatives exist.

Doc2Vec offers a clear improvement over Word2Vec by generating document level vectors directly. However, its performance heavily depends on the size of the training set, often requiring large datasets to produce robust representations. Moreover, when adding new documents (or products), it will require a retraining.

In contrast, Sentence2Vec provides more expressive embeddings for short texts, as it is specifically designed to capture the overall meaning of a sentence. It also requires less data to achieve strong performance, although this comes at the cost of higher computational requirements compared to Doc2Vec.

Overall, for similarity and retrieval tasks such as our product ranking, Sentence2Vec is generally a more effective choice than Doc2Vec, provided that the additional computational cost is acceptable.

3.2. Tag of the repository

This report references the notebook in:
https://github.com/IsaacPerez02/SearchEngine-IRWA_2025/releases/tag/IRWA-2025-part-3